

*producer consumer.c X

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #include <semaphore.h>
5
6  #define BUFFER_SIZE 5
7
8  int buffer[BUFFER_SIZE];
9  int in = 0, out = 0, item = 0;
10
11 sem_t empty, full;
12 pthread_mutex_t mutex;
13
14 void produce() {
15     sem_wait(&empty);
16     pthread_mutex_lock(&mutex);
17
18     item++;
19     buffer[in] = item;
20     printf("Producer has produced: Item %d\n", item);
21     in = (in + 1) % BUFFER_SIZE;
22
23     pthread_mutex_unlock(&mutex);
24     sem_post(&full);
25 }
26
27 void consume() {
28     sem_wait(&full);
29     pthread_mutex_lock(&mutex);
30
31     int consumed = buffer[out];
32     printf("Consumer has consumed: Item %d\n", consumed);
33     out = (out + 1) % BUFFER_SIZE;
34
35     pthread_mutex_unlock(&mutex);
36     sem_post(&empty);
37 }
38
39 int main() {
40     int choice;
41
42     sem_init(&empty, 0, BUFFER_SIZE);
43     sem_init(&full, 0, 0);
44     pthread_mutex_init(&mutex, NULL);
45
46     while (1) {
```

```

46 while (1) {
47     printf("\nEnter 1. Producer 2. Consumer 3. Exit\n");
48     printf("Enter your choice: ");
49     scanf("%d", &choice);
50
51     switch (choice) {
52         case 1:
53             produce();
54             break;
55
56         case 2:
57             if (sem_trywait(&full) == -1) {
58                 printf("Buffer is empty!\n");
59             } else {
60                 pthread_mutex_lock(&mutex);
61                 int consumed = buffer[out];
62                 printf("Consumer has consumed: Item %d\n", consumed);
63                 out = (out + 1) % BUFFER_SIZE;
64                 pthread_mutex_unlock(&mutex);
65                 sem_post(&empty);
66             }
67             break;
68
69         case 3:
70             sem_destroy(&empty);
71             sem_destroy(&full);
72             pthread_mutex_destroy(&mutex);
73             exit(0);
74
75         default:
76             printf("Invalid choice!\n");
77     }
78 }
79
80 return 0;
81 }
82

```

```
"C:\Users\Admin\Desktop\1w  X + v
Enter 1. Producer 2. Consumer 3. Exit
Enter your choice: 1
Producer has produced: Item 1

Enter 1. Producer 2. Consumer 3. Exit
Enter your choice: 2
Consumer has consumed: Item 1

Enter 1. Producer 2. Consumer 3. Exit
Enter your choice: 1
Producer has produced: Item 2

Enter 1. Producer 2. Consumer 3. Exit
Enter your choice: 2
Consumer has consumed: Item 2

Enter 1. Producer 2. Consumer 3. Exit
Enter your choice: 1
Producer has produced: Item 3

Enter 1. Producer 2. Consumer 3. Exit
Enter your choice: 2
Consumer has consumed: Item 3

Enter 1. Producer 2. Consumer 3. Exit
Enter your choice: 3

Process returned 0 (0x0)   execution time : 9.829 s
Press any key to continue.
|
```

producer consumer.c X *dining philosopher.c X

```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include <unistd.h>
4
5  #define N 5 // Number of philosophers
6
7  pthread_mutex_t forks[N]; // One mutex per fork
8  pthread_t philosophers[N];
9
10 // Philosopher function
11 void* philosopher(void* num) {
12     int id = *(int*)num;
13
14     int left = id;           // left fork index
15     int right = (id + 1) % N; // right fork index
16
17     while (1) {
18         printf("Philosopher %d is thinking.\n", id);
19         sleep(1); // Thinking
20
21         // Deadlock avoidance strategy
22         if (id % 2 == 0) {
23             // Even philosopher picks left then right
24             pthread_mutex_lock(&forks[left]);
25             printf("Philosopher %d picked up left fork %d.\n", id, left);
26
27             pthread_mutex_lock(&forks[right]);
28             printf("Philosopher %d picked up right fork %d.\n", id, right);
29         } else {
30             // Odd philosopher picks right then left
31             pthread_mutex_lock(&forks[right]);
32             printf("Philosopher %d picked up right fork %d.\n", id, right);
33
34             pthread_mutex_lock(&forks[left]);
35             printf("Philosopher %d picked up left fork %d.\n", id, left);
36         }
37
38         // Eating
39         printf("Philosopher %d is eating.\n", id);
40         sleep(2);
41
42         // Put down forks
43         pthread_mutex_unlock(&forks[left]);
44         pthread_mutex_unlock(&forks[right]);
45
46         printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
47     }
48
49     return NULL;
50 }
```

```
48
49     return NULL;
50 }
51
52 int main() {
53     int i;
54     int ids[N];
55
56     // Initialize forks (mutexes)
57     for (i = 0; i < N; i++) {
58         pthread_mutex_init(&forks[i], NULL);
59         ids[i] = i;
60     }
61
62     // Create philosopher threads
63     for (i = 0; i < N; i++) {
64         pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
65     }
66
67     // Join threads (runs forever)
68     for (i = 0; i < N; i++) {
69         pthread_join(philosophers[i], NULL);
70     }
71
72     // Destroy mutexes (not reached)
73     for (i = 0; i < N; i++) {
74         pthread_mutex_destroy(&forks[i]);
75     }
76
77     return 0;
78 }
79
```

producer consumer.c X dining philosopher.c X

```
31
32
33
34 Philosopher 0 is thinking.
35 Philosopher 4 is thinking.
36 Philosopher 1 is thinking.
37 Philosopher 2 is thinking.
38 Philosopher 3 is thinking.
39 Philosopher 1 picked up right fork 2.
40 Philosopher 1 picked up left fork 1.
41 Philosopher 1 is eating.
42 Philosopher 3 picked up right fork 4.
43 Philosopher 3 picked up left fork 3.
44 Philosopher 3 is eating.
45 Philosopher 0 picked up left fork 0.
46 Philosopher 0 picked up right fork 1.
47 Philosopher 0 is eating.
48 Philosopher 3 put down forks 3 and 4.
49 Philosopher 3 is thinking.
50 Philosopher 4 picked up left fork 4.
51 Philosopher 1 put down forks 1 and 2.
52 Philosopher 1 is thinking.
53 Philosopher 2 picked up left fork 2.
54 Philosopher 2 picked up right fork 3.
55 Philosopher 2 is eating.
56 Philosopher 1 picked up right fork 2.
57 Philosopher 1 picked up left fork 1.
58 Philosopher 1 is eating.
59 Philosopher 4 picked up right fork 0.
60 Philosopher 4 is eating.
61 Philosopher 0 put down forks 0 and 1.
62 Philosopher 0 is thinking.
63 Philosopher 2 put down forks 2 and 3.
64
65
66
67
```